

Here is a detailed, technical breakdown of why you were blocked by the local environment settings and how your architecture has evolved with the shift to the hosted API. You can use these exact structural points to document your final year project or explain it during your resume interviews.

Part 1: Why Local Docker Setup Failed (The Microsoft/WSL2 Block)

When you tried to run Judge0 locally inside Docker on Windows, the system repeatedly crashed with the error: Failed to create control group /sys/fs/cgroup/memory/box-5/: No such file or directory.

This failure happened due to a low-level architectural conflict between the Judge0 sandboxing tool (isolate) and Microsoft's WSL2 (Windows Subsystem for Linux) kernel settings:

1. The Legacy Cgroup v1 Dependency

Judge0 relies internally on a Linux kernel feature called **Control Groups (cgroups)** to enforce strict memory caps, CPU limits, and process isolation. Specifically, Judge0's underlying engine requires **cgroup v1**. In a cgroup v1 architecture, the Linux kernel creates isolated directories for separate system resources (such as /sys/fs/cgroup/memory/).

2. Modern WSL2 Defaults (Cgroup v2)

Modern Linux distributions and the current Microsoft WSL2 virtual kernel have completely transitioned to **cgroup v2** (a unified hierarchy where resources are not split into distinct sub-folders like memory/). When Judge0 launched inside Docker, it immediately looked for the path /sys/fs/cgroup/memory/ to write resource limits for the student's code execution. Because your WSL2 environment is strictly running cgroup v2, that path literally did not exist, causing the engine to crash.

3. The Microsoft Kernel Constraint

On a native Linux machine, an engineer can easily resolve this by editing the GRUB bootloader parameters to force the system to boot into legacy cgroup v1. However, **WSL2 does not use GRUB**.

- Microsoft compiles its own proprietary, highly optimized virtual Linux kernel.
- In recent kernel updates, Microsoft explicitly compiled the kernel with CONFIG_MEMCG_V1 **disabled** at the source code level to prioritize cgroup v2 efficiency.
- Consequently, your Windows Docker setup was physically incapable of allocating legacy cgroup v1 folders, forcing you to move the execution environment away from your local host machine.

Part 2: Are We Still Using Redis Queues & Asynchronous Webhooks?

Yes, absolutely. Shifting to the cloud API did not eliminate your backend infrastructure; it only changed who executes the untrusted code. Your platform still strictly depends on **Redis (via**

BullMQ) and **Asynchronous Webhooks** to remain highly performant and scalable. Here is how the new architecture functions and how it structurally differs from your previous "fully local" plan:

How it is Different Now

Component	Previous Local Architecture	Current Cloud-Hybrid Architecture
Primary Redis Queue (BullMQ)	Managed the raw pacing of your internal servers so they didn't freeze up trying to open thousands of local Docker sandboxes simultaneously.	Now acts as a rate-limiting and reliability buffer to protect your limited RapidAPI credit quota, ensuring concurrent user clicks don't burst past your cloud tier thresholds.
Code Isolation	Handled locally on your machine via Dockerized containers using host system RAM and CPU.	Completely offloaded to Judge0's remote multi-tenant secure cloud infrastructure.
The Webhook Pipeline	The local worker made an internal HTTP call (localhost:2358) and waited directly for the response or handled local callback pings.	The worker dispatches the job payload safely to the cloud, and the remote Judge0 cloud server hits your exposed public-facing endpoint (APP_WEBHOOK_URL) once execution terminates.

The New Asynchronous Lifecycle Step-by-Step

1. **Submission Entry:** When a student hits "Submit," your Next.js server instantly writes a QUEUED record to PostgreSQL and pushes the task metadata into your local **Redis Queue (BullMQ)**. The user's browser immediately receives a 202 Accepted status—the UI does not hang.
2. **Worker Handoff:** Your backend BullMQ worker pulls the job from your local Redis instance asynchronously. It compiles a clean JSON payload, encodes the code strings into **Base64 formats** (to protect against line breaks crashing the parser), and sends a fast outbound HTTP POST request to `[https://judge0-ce.p.rapidapi.com/submissions?base64_encoded=true](https://judge0-c`

e.p.rapidapi.com/submissions?base64_encoded=true).

3. **The Webhook Loop:** Instead of sitting there stalling your worker thread waiting for C++ or Python code to run, the worker registers your server's address (APP_WEBHOOK_URL) in the request and instantly frees itself to pick up the next student's submission.
4. **Cloud Callback:** The Judge0 cloud processes the compilation safely in their remote infrastructure. Once finalized, their servers asynchronously send a POST webhook request back to your application server containing the raw performance statistics (Execution time, Memory used, AC/WA/TLE status codes).
5. **Real-Time Sync:** Your server receives the webhook payload, updates PostgreSQL, and broadcasts the final outcome over your **Socket.IO Redis Adapter** straight to the student's browser.

By adopting this cloud-hybrid approach, your resume project looks incredibly strong: you successfully engineered a distributed backend architecture that cleanly bridges local messaging queues with third-party cloud computing patterns!